

```

    question = "What language did you first learn to speak?"
    language_survey = AnonymousSurvey(question)
    return language_survey

❸ def test_store_single_response(language_survey):
    """Test that a single response is stored properly."""
❹     language_survey.store_response('English')
    assert 'English' in language_survey.responses

❺ def test_store_three_responses(language_survey):
    """Test that three individual responses are stored properly."""
    responses = ['English', 'Spanish', 'Mandarin']
    for response in responses:
❻         language_survey.store_response(response)

    for response in responses:
        assert response in language_survey.responses

```

We need to import `pytest` now, because we're using a decorator that's defined in `pytest`. We apply the `@pytest.fixture` decorator ❶ to the new function `language_survey()` ❷. This function builds an `AnonymousSurvey` object and returns the new survey.

Notice that the definitions of both test functions have changed ❸ ❺; each test function now has a parameter called `language_survey`. When a parameter in a test function matches the name of a function with the `@pytest.fixture` decorator, the fixture will be run automatically and the return value will be passed to the test function. In this example, the function `language_survey()` supplies both `test_store_single_response()` and `test_store_three_responses()` with a `language_survey` instance.

There's no new code in either of the test functions, but notice that two lines have been removed from each function ❹ ❻: the line that defined a question and the line that created an `AnonymousSurvey` object.

When we run the test file again, both tests still pass. These tests would be particularly useful when trying to expand `AnonymousSurvey` to handle multiple responses for each person. After modifying the code to accept multiple responses, you could run these tests and make sure you haven't affected the ability to store a single response or a series of individual responses.

The structure above will almost certainly look complicated; it contains some of the most abstract code you've seen so far. You don't need to use fixtures right away; it's better to write tests that have a lot of repetitive code than to write no tests at all. Just know that when you've written enough tests that the repetition is getting in the way, there's a well-established way to deal with the repetition. Also, fixtures in simple examples like this one don't really make the code any shorter or simpler to follow. But in projects with many tests, or in situations where it takes many lines to build a resource that's used in multiple tests, fixtures can drastically improve your test code.

When you want to write a fixture, write a function that generates the resource that's used by multiple test functions. Add the `@pytest.fixture`